Step 4 — A Fallback When a Server Is Unreachable, and a Full Working Demo — Hints

Back to the step · Hint 1 · Hint 2 · Solution

- Hint 1 — The idea, and the exact pieces
- Hint 2 — The plan, and almost the whole thing

Hint 1 — The idea, and the exact pieces

Resilience wraps around the existing pool and loop — it doesn't rewrite them. Three guards:

- **A server that won't connect** — wrap each server in `connect_all` in `try/except`; log `"[name] unreachable: …, continuing without it"` and `continue`. The others still connect.
- **A server that never finishes starting** — wrap `session.initialize()` in `asyncio.wait_for(..., timeout=self.connect_timeout)`. A hang becomes a `TimeoutError`, caught by the same `try/except`.
- **A tool call that fails mid-task** — wrap the `call_tool` dispatch in `try/except`; return a clear `"Error: …"` string as the observation (Doc06's "get the failure back to the model correctly"), not a crash.

And the loop already raises `MaxIterationsExceeded` at its cap (Step 2) — keep it; a confusing failure is exactly what makes a model retry forever.

Hint 2 — The plan, and almost the whole thing

The one trap that will cost you an hour if you miss it: do NOT write `await asyncio.wait_for(self._connect_one(server), ...)`. `wait_for` runs its coroutine in a **new task**, so the `stdio_client` / `ClientSession` contexts get entered in that task — and `aclose()` later tries to exit them in the main task, raising `RuntimeError: Attempted to exit cancel scope in a different task`. Enter the contexts in the main task; time out only `initialize()`.

```
server_pool._connect_one(server):
    rd, wr  = await stack.enter_async_context(stdio_client(params))  # main task
    session = await stack.enter_async_context(ClientSession(rd, wr))
    await asyncio.wait_for(session.initialize(), timeout=connect_timeout)
```

```
    return session

server_pool.connect_all():
    for server:
        try: session = await self._connect_one(server)
        except Exception as e:
            log "[name] unreachable: {e}, continuing without it"; continue
        ...discover + namespace as before...

server_pool.call_tool(namespaced, args):
    if namespaced not in routing: return f"Error: no tool named {namespaced}"
    try: result = await session.call_tool(tool, args)
    except Exception as e: return f"Error: tool {namespaced} failed: {e}"
    return extract_text(result)

main.py:  the full demo — print connected servers, run the both-servers task
test_agent.py (+ case 4):
    connect([NOTES, BROKEN]); assert "broken" not in pool.sessions
    assert "notes" in pool.sessions; run a task; assert out.answer
```

The whole final file set, runnable, is in the Solution.